Institut für Informatik
Prof. Dr. Heiko Röglin
Joshua Könen,
Sarah Sturm,
Aaron Weinmann

UNIVERSITÄT BONN

Lab: Efficient Algorithms
for selected Problems
Summer 2025

Problem Set 5

Tasks that are marked with * are optional for groups with only two members and will not count into their score.

1 Skyline*

Task: You are a famous architect and your current project is to plan an entire new city. Since you are impressed by Manhattan you decide that the streets will form a grid, and in each block there shall be a gigantic skyscraper. Investors already handed in plans for the buildings. You are interested in the minimum distance between two skyscrapers of the same height, because this is important to ensure that the city will have a nice skyline.

Input: The blocks of the city are arranged in a grid of dimension n. The first line of the input contains this number. Then n lines follow. The i-th line contains the n heights h_{ij} of the skyscrapers in row i.

You can assume without loss of generality that the heights are in $\{0, \dots, n^2 - 1\}$ and that there are at least two skyscrapers of the same height. The distance between two blocks is given by the Manhattan distance.

Output: Output the minimum distance between two skyscrapers of the same height.

Sample Input 1:	**Sample Output 1:**	**Sample Input 2:**	**Sample Output 2:**
3 0 1 0 1 0 1 0 0 0	1	4 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 0	6

2 Sun Tzu

Sun Tzu does what he knows best, he fights a war. The famous strategist knows that there is no way he can win every battle, but he wants to maximize his odds of winning the war. Sun Tzu has s soldiers and the opposing general has s' soldiers. There are k battlefields. If a side sends more soldiers to a battlefield that side wins the battle. If one side wins more battles than the other, it wins the war.

Please note that the problem is posed this way for historic reasons. Currently it is more relevant for marketing, elections and strategic planning in economy.

Hint: This problem is easier to solve in C++ or Java, since there are some possible solutions that take too much time in Python.

Input: The input consists of number of soldiers on both sides s and s' and the number of battlefields k. Each number is given as an integer.

Output: The ratio v of wars won by Sun Tzu if he chooses the best deterministic strategy and faces a general that picks his strategy uniformly at random. Please return the result as a floating point number up to 3 digits of precision.

Sample Input 1:	**Sample Output 1:**	**Sample Input 2t:**	**Sample Output 2:**
2 1 2	1	6 6 3	0.429

In the first instance the strategy $(1, 1)$ *always wins one battlefield and ties on the other. Note that* $(2, 0)$ *only achieves a draw in half the cases.*

3 Apples

For the annual meeting of the Jane Austen book club, you are supposed to buy exactly n apples as a snack. The supermarket sells apples in two packaging sizes: You can buy a apples for c_a Euros, or b apples for c_b Euros. How many of each package should you buy to minimize your cost and get exactly n apples?

Input: The input specifies multiple test cases. Each starts with a line containing n. The second line contains at first the costs c_a and then the number of apples a, the third line contains at first the costs c_b and then the number of apples b. All numbers are integers. The file ends with a line containing 0.

Output: For each test case, print a line. If there is a solution, print the number of packages of the first type that you should by first, then print the number of packages of the second type that you should buy. If no solution exists, output `failed`.

Sample Input:	**Sample Output:**

`24`
`4 2`
`6 2`
`11`
`4 2`
`6 2`
`0`
	`12 0`
	`failed`

4 Sweet Tooth

You are preparing giveaway presents for your child's birthday party. For this, you have bought ridiculous amounts of various sweets. Now you want to pack a present for each of the children. It is mandatory that each child receives the same amount of sweets to prevent grievance. Thus, you do this: Out of the s sweets, you make c piles for each of the c children, and each of these piles contains $\lfloor s/c \rfloor$ sweets, and then you eat the remaining $s - c \cdot \lfloor s/c \rfloor$ yourself. With this general plan in mind, you set out to make customized piles for all children since each child prefers different sweets.

Input: The first line contains c, the number of children. The second line contains t, the number of different types of sweets. The third line contains t numbers, the ith number says how many sweets of type i you have (this means that s is the sum of the numbers in the third line).

The following c lines each contain t numbers between 1 and c which indicate the preferences of the children for each type of sweet (a lower number indicates that the sweet is higher in the priority list; priorities can be used multiple times).

Output: Compute the minimum p such that it is possible to cluster the sweets into c piles for

the children and one pile for yourself such that each child receives $\lfloor s/c \rfloor$ sweets with a priority of $\leq p$.

Sample Input:

```
5
5
1 1 1 1 1
1 2 3 4 5
2 3 4 5 1
1 1 1 1 1
2 2 2 3 5
5 4 3 2 1
```

Sample Input:

```
2
```

5 Duckburg*

Scrooge McDuck (german: Dagobert Duck) runs a new lottery. To participate, players have to pay 25 Dollar. Then each participant chooses an integer between 1 and 1000 and gives it to Scrooge. After all participants have chosen, Scrooge chooses i integers between 1 and 1000. Now all participants get their winnings. For every participant, the closest 'Scrooge number' to the participant's number is determined. The absolute difference between the two numbers is what the participant gets in Dollar.

Write a program that maximizes Scrooge's winnigs / minimizes his losses.

Input: The first line contains $i \leq 100$, the second line contains $n \leq 1000$. Then n lines containing a number between 1 and 1000 each, which is the number that the participant chose. You can assume that all participants choose different numbers.

Output: An integer which is Scrooge's net total if he chooses his numbers optimally. The number is positive if he wins money and negative if he loses money.

Note: We ask for an optimal solution here. If we see a heuristic, we might upload new testcases into the running contest.

Sample Input:

```
10
11
1
2
3
4
5
6
7
8
9
10
11
```

Sample Output:

```
274
```

6 Sheep

Task: Shepherd Ellie wants to buy pastures for their sheep. Due to a surprising return of wolves to the surrounding forest, a lot of pastures are for sale. However, to avoid making the same mistakes as the previous shepherds, Ellie needs to build a fence around the area they are going to buy. Ellie has their eyes set on a few pastures that they absolutely want to purchase, but it might also be profitable to buy additional neighboring pastures to reduce the overall perimeter of the area and thus decrease the cost of the fence. You know the cost of each pasture, the total cost of building a fence around each pasture and the cost of the shared border of neighbored pastures. Calculate the minimum cost required to buy and fence in all the required areas.

Input: The input has the following form:

- The first line contains three integers n, r and m. n is the number of pastures for sale, r is the number of required pastures and m is the number of neighboured pasture pairs.
- The second line contains r integers, the indices of the required pastures.
- The following n lines contain two integers each, a_i is the cost to buy pasture i, b_i is the cost to build a fence around pasture i
- The next m lines contain three integers each, i, j and b_{ij}. i and j are two neighboring pastures and b_{ij} is the cost to build a fence on their shared border. You may assume that $\sum_j b_{ij} \leq b_i$ with the possibility of some part of the border of i being adjacent to the forest.

Output: Compute the minimum amount of money needed to buy all of the required areas and build a fence around them. The output is a single integer c. Again, note that it might be beneficial to buy additional non-required pastures to decrease the overall cost.

Sample Input 1:

```
4 2 3
0 2
2 5
1 8
3 6
9 15
0 1 4
1 2 3
2 3 1
```

Sample Output 1:

```
11
```

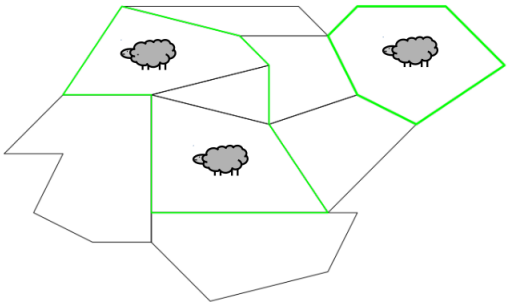

Figure 1: Buying the triangular pasture is not necessary but can reduce the overall perimeter and thus the total cost

7 Lawn-mowing Robots

A well known company in Silicon Valley has developed a new lawn-mowing robot. Unfortunately, the battery is not quite perfect, therefore every model also comes with three photo-voltaic charging stations that are used to reload the battery.

You have bought such a robot and want it to mow your rectangular lawn, which is subdivided into cells and looks like a grid. You have installed the charging stations at three of the grid cells, and their position cannot be changed.

To prevent the grass from developing ugly marks and stripes, you want to program your robot in such a way that it uses a different mowing pattern every day. In every such pattern, the robot should visit every grid cell only once and it should stop at the charging stations after a $\frac{1}{4}$, $\frac{1}{2}$ and $\frac{3}{4}$ fraction (rounded down) of the whole way. The robot is quite a new product with a few strange quirks and has to visit its loading station exactly in a given order.

Hint:

We are not aware of a solution in Python that adheres to the time limit. Therefore we recommend using C++ or Java.

Input:

The input begins with a line containing two integers r and c, where $2 \leq r, c \leq 8$, specifying the number of rows and columns, respectively, in the grid. This is followed by a line containing six integer values r_1, c_1, r_2, c_2, and r_3, c_3, where $0 \leq r_i < r$ and $0 \leq c_i < c$ for $i = 1, 2, 3$. These are the positions of the three charging stations.

Output:

Print the number of possible tours that begin at row 0, column 0, end at row 0, column 1, and visit row r_i , column c_i at time $\lfloor \frac{i}{4} \cdot r \cdot c \rfloor$ for $i = 1, 2, 3$.

Sample Input 1:	**Sample Output 1:**	**Sample Input 2:**	**Sample Output 2:**
3 6 2 1 2 4 0 4	2	4 3 2 0 3 2 0 2	0

8 Area of Intersection of Polygons

You are given several simple polygons. Calculate the area of the region where all polygons intersect, allowing an absolute error of at most 0.05.

Hint: Although the task is technically solvable in Python, we strongly recommend using c++

or java, since the time limit is quite tight for python.

Input: The first line of the input contains the number of polygons n. Then n lines follow. Each line i starts with the number of corners k_i of polygon i followed by k_i pairs of numbers a_{i,k_i}, b_{i,k_i} that describe the coordinates of each corner. The polygon is constructed by connecting consecutive corners by lines and connecting the last corner to the first corner by a line.

You can assume that the coordinates of the corners are integers, and that it holds true that $0 \leq a_{i,k_i}, b_{i,k_i} \leq 10$.

Output: The size of the area where all n polygons intersect with an error of at most 0.05.

Sample Input:

```
2
4 1 1 4 1 4 4 1 4
3 2 0 6 0 4 2
```

Sample Output:

```
0.5
```